HOVI MOUSE

Submitted in partial fulfilment of the requirements for the award of the Degree of

Bachelor of Science in Computer Science

Submitted By
Dharshini priya V G (23UCS20)
Hamsathvani K (23UCS21)

Under the Guidance of
Dr. M.R.SUMATHI MCA.,M.Phil.,Ph.D.,



DEPARTMENT OF COMPUTER SCIENCE

THIAGARAJAR COLLEGE (AUTONOMOUS)
(Affiliated to Madurai Kamaraj University)
Re-accredited with 'A++' Grade by NAAC
Ranked 20th in NIRF 2025
Madurai – 625009

October/November– 2025

THIAGARAJAR COLLEGE (AUTONOMOUS)
(Affiliated to Madurai Kamaraj University)
Re-Accredited with 'A++' Grade by NAAC
Ranked 20th in NIRF 2025

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that **"HOVI MOUSE"** is a bonafide project work done by **Dharshini Priya V G (23UCS20)** and **Hamsathvani K (23UCS21)** of B.Sc Computer Science during the fifth semester. Submitted for the final viva-voce held on _____

Project Guide
(Dr. M.R.SUMATHI)

Head of the Department
(Mrs.SM.VALLI)

External Examiner

Dharshini Priya V G(23UCS20)
B. Sc Computer Science
Department of Computer Science
Thiagarajar College (Autonomous)
Madurai - 625009

DECLARATION

Hereby, I declare that the project work entitled ***"HOVI MOUSE",*** submitted to Thiagarajar College, Madurai, has been carried out by me as a part of the curriculum. To the best of my knowledge, this is my original work and has not been copied from any other source. This project work has not been submitted to any other University/Institute for the award of any degree.

Place: **Signature**

Date: (Dharshini Priya V G)

(23UCS20)

Hamsathvani K (23UCS21)
B.Sc Computer Science
Department of Computer Science
Thiagarajar College (Autonomous)
Madurai - 625009

DECLARATION

Hereby, I declare that the project work entitled **"HOVI MOUSE",** submitted to Thiagarajar College, Madurai, has been carried out by me as a part of the curriculum. To the best of my knowledge, this is my original work and has not been copied from any other source. This project work has not been submitted to any other University/Institute for the award of any degree.

Place:

Date:

Signature

(Hamsathvani K)

(23UCS21)

ACKNOWLEDGEMENT

We extend our sincere thanks to the Almighty for blessing us with the physical strength, mental ability, and determination that enabled us to complete this project effectively.

We are grateful to our respected Principal, **Dr. D. PANDIARAJA, M.Sc., M.Phil., P.G.D.C.A., B.Ed., Ph.D., Thiagarajar College, Madurai**, for granting us the opportunity to carry out this project work under the esteemed guidance of our department.

We wish to express our profound gratitude to the Head of the Department of Computer Science, **Mrs. SM. VALLI, B.E., M.C.A., M.Phil.** We sincerely thank her for her valuable guidance, constant encouragement, and support throughout the progress of this work.

We are especially indebted to our Project Guide, **Dr. M. R. SUMATHI, M.C.A., M.Phil., Ph.D.**, for her patient guidance, constructive suggestions, and for having faith in our ability. Her keen interest and valuable feedback at every stage greatly contributed to the successful completion of this project.

We also extend our heartfelt thanks to all the faculty members of the Department of Computer Science for their constant support, academic guidance, and encouragement during the course of our project.

Finally, we express our deep sense of gratitude to our family members and friends for their unwavering encouragement, moral support, and understanding, which have been a pillar of strength throughout our work.

CONTENTS

Sl. No.	Title	Pg. No.
1	**INTRODUCTION** 1.1 Abstract 1.2 Project Description	 **2** **3**
2	**SYSTEM ANALYSIS** 2.1 Existing System 2.2 Proposed System	 **7** **8**
3	**SYSTEM CONFIGURATION** 3.1 Hardware Specification 3.2 Software Specification 3.3 About The Language	 **11** **11** **11**
4	**SYSTEM DESIGN** 4.1 Data Flow Description 4.2 Module Description	 **16** **22**
5	**SYSTEM IMPLEMENTATION** 5.1 Sample coding 5.1.1 Voice Command 5.1.2 Gesture coding	 **28** **28** **33**
6	**SYSTEM TESTING** 6.1 System Testing 6.2 Types of Testing	 **45** **45**
7	**SCREENSHOTS**	**53**
8	**CONCLUSION**	**59**
9	**FUTURE ENHANCEMENTS**	**61**
	BIBLIOGRAPHY	**63**

INTRODUCTION

1. INTRODUCTION

1.1 ABSTRACT

HOVI Mouse is an innovative virtual mouse system that enables complete computer interaction through hand gestures and voice commands. It integrates technologies such as OpenCV and MediaPipe for real-time gesture tracking, along with SpeechRecognition, PyAutoGUI, and pyttsx3 for accurate voice-controlled operations. The project focuses on replacing traditional mouse usage with a smart, contactless solution that supports everyday computing needs.

Voice command module in HOVI Mouse supports basic mouse events and application control, enabling users to operate the system more efficiently and reduce reliance on traditional mouse and keyboard.

In practice, this system reduces reliance on mouse and keyboard by enabling common actions through hands-free voice or gesture. It combines gesture-based navigation with voice commands for typing, handling, and menu control, offering a complete alternative to conventional input. Designed with noise reduction, the recognition engine ensures reliable real-life performance, directly executing recognized actions for smooth and efficient interaction.

The HOVI Mouse is highly useful for differently-abled users and professionals seeking hands-free control in situations where traditional devices are inconvenient, offering accessibility, reduced hardware dependency, and innovative human-computer interaction. It simplifies daily operations by integrating gesture and voice-based control for smoother user experience.

1.2 PROJECT DESCRIPTION

The project aims to develop an innovative virtual mouse system, HOVI Mouse, that enables hands-free computer interaction through voice commands and hand gestures. It is designed to assist differently-abled users, professionals, and individuals seeking convenience where traditional input devices like the mouse and keyboard are inconvenient. The system improves accessibility, reduces dependency on hardware devices, and promotes smarter human-computer interaction.

The system integrates MediaPipe for gesture recognition and SpeechRecognition for accurate voice control, offering reliable mouse events, application control, and efficient navigation. By combining these technologies, the project provides a seamless alternative to conventional input devices, making interaction simple, direct, and efficient.

Implementation Steps:

1. Gesture Recognition

Gesture recognition is a computer vision technology that interprets human hand movements and finger positions as input commands through real-time video analysis. It uses machine learning algorithms to detect, track, and classify hand poses into meaningful control signals for computer interaction.

Required Libraries

- OpenCV
- Mediapipe
- Numpy
- Math

Implementation in HOVI Mouse

The gesture recognition module captures hand movements through webcam input, detects 21 hand landmarks using MediaPipe, and classifies finger positions into specific gestures like cursor movement (index finger), left click (thumb + index), right click (thumb + middle), scrolling gestures, and copy/paste commands.

2. Voice Command Module

A voice command module is a speech recognition system that converts spoken language into text commands, processes natural language input, and executes corresponding system actions. It enables hands-free computer control through audio input processing and command interpretation.

Required Libraries

- SpeechRecognition
- pyAudio
- Threading

Implementation in HOVI Mouse

The voice command module continuously listens for spoken commands, processes natural language phrases like "open chrome," "left click," or "enable gesture mode," and executes corresponding system functions while maintaining simultaneous operation with gesture recognition.

3. Integration Layer

An integration layer is a software architecture component that combines multiple input modalities into a unified control system. It manages the coordination between different input methods, resolves conflicts, and provides seamless switching between control modes.

Required Libraries

- Threading
- Time
- Collections (deque)

Implementation in HOVI Mouse

The integration layer coordinates between gesture and voice inputs, allows users to enable/disable gesture mode through voice commands, maintains both control methods simultaneously, and ensures proper resource management and thread synchronization.

4. Recognition Engine

A recognition engine is a processing system that filters, analyzes, and validates input signals to ensure accurate command detection while minimizing false positives. It implements noise reduction, signal stabilization, and confidence-based decision making for reliable real-world performance.

Implementation in HOVI Mouse

The recognition engine implements gesture stability checking through buffered frame analysis, applies smoothing algorithms for cursor movement, manages action cool downs to prevent accidental triggers, and provides noise-resistant voice command processing with ambient noise calibration.

5. Accessibility Focus

Accessibility focus in assistive technology refers to designing inclusive interfaces that accommodate users with diverse physical abilities and needs. It emphasizes intuitive interaction methods, customizable sensitivity settings, and alternative input modalities for enhanced usability.

Implementation in HOVI Mouse

The accessibility focus is implemented through multiple redundant control methods (voice + gesture), customizable gesture sensitivity, comprehensive application control through voice commands, visual feedback through on-screen displays, and the ability to switch between input modes based on user capabilities and preferences.

SYSTEM ANALYSIS

2. SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

Currently, Computer systems rely mainly on traditional input devices such as mouse and keyboard for navigation and control. Some accessibility tools provide limited features like on-screen Keyboards or basic voice commands, but they lack gesture recognition and advanced voice - controlled interactions. Existing solutions are not fully integrated to offer seamless hands-free control, making them less efficient for differently-abled users or multitasking professionals.

DISADVANTAGES

1. No Gesture-Based Control

Current computer systems lack the ability to interpret hand movements and finger positions as input commands, forcing users to rely exclusively on physical hardware devices for all interactions.

Real-World Example: A student cannot point at the screen to move the cursor. They must always use a physical mouse to click on files or open programs.

2. Limited and Inaccurate Voice Command Support

Existing voice recognition systems offer basic functionality with poor accuracy rates, restricted vocabulary, and inability to handle complex natural language commands or environmental noise interference.

Real-World Example: When someone says "open Word" to their computer, it often does not understand or opens the wrong program. Users must spell words slowly or use the keyboard instead.

3. No Advanced System-Level Navigation

Current accessibility tools cannot handle complex interface elements such as dropdown menus, window management, context menus, or multi-layered system operations through alternative input methods.

Real-World Example: Users cannot right-click using voice commands to copy files or access menu options. They must physically use the mouse for these basic tasks.

4. Not User-Friendly for Real-Time Multitasking and Accessibility

Existing systems lack seamless integration between different input modalities and cannot support efficient concurrent operations through hands-free methods, creating fragmented user experiences.

Real-World Example: A teacher presenting lessons cannot scroll through slides while talking to students. They must stop speaking to use the mouse, interrupting their presentation flow.

2.2 PROPOSED SYSTEM

The proposed system aims to develop an advanced gesture and voice-controlled virtual mouse (HOVI-MOUSE) that provides hands-free, real-time interactions with computers. Unlike traditional input devices, this system integrates openCV, mediapipe, speechRecognition, pyAutoGUI, and Keyboard libraries to deliver seamless navigation and accessibility.

It provides:

1. Gesture-Based Mouse Control

Gesture-based mouse control lets users move the cursor, click, drag, and scroll using hand movements detected by a webcam. Specific gestures like pointing or pinching are mapped to mouse actions, providing a touchless and intuitive way to interact with the computer.

2. Voice Commands for System-Level Actions

Voice commands allow users to perform actions such as right-click, double-click, open applications, or press keyboard keys by speaking. The system converts speech to text and executes commands instantly, enabling hands-free operation and efficient control.

3. Dropdown and Window Navigation

Dropdown menus and windows can be navigated using voice-controlled keyboard shortcuts. Complex sequences that usually require multiple clicks or keystrokes can now be performed with simple spoken commands, saving time and effort.

4. Background Noise Handling and Casual Speech Recognition

The system filters ambient noise and recognizes natural speech variations, ensuring accurate command detection even in noisy environments. Users can speak casually without repeating instructions, making interactions smoother and faster.

5. Accessibility and Multitasking Efficiency

HOVI-MOUSE supports differently-abled users and enhances multitasking for professionals. Hands-free control through gestures and voice allows users to perform tasks efficiently while reducing dependency on traditional input devices.

ADVANTAGES

The HOVI-MOUSE provides a completely hands-free computer control system, allowing users to operate the computer without touching a keyboard or mouse. This makes interaction more natural and reduces physical strain.

By combining gesture and voice inputs, the system supports multitasking, enabling users to perform other activities while controlling the computer. This enhances efficiency in professional and daily tasks.

The system improves accessibility for differently-abled users, allowing them to navigate and control applications easily. Voice and gesture commands remove the dependency on traditional input devices.

HOVI-MOUSE also enables faster and smoother navigation compared to conventional devices. Cursor movement, clicks, scrolling, and window management can be performed quickly through intuitive gestures and spoken commands.

- Provides a hands-free computer control system.
- Supports multitasking with combined gesture and voice input.
- Improve accessibility for differently-abled users.
- Enables faster and smoother navigation compared to traditional devices
- Offers user-friendly interaction with real-time responsiveness

SYSTEM CONFIGURATION

3. SYSTEM CONFIGURATION

3.1. HARDWARE SPECIFICATIONS

Processor	: Intel(R) Core(TM)
Processor Speed	: i5-1235U 1.30 GHz
RAM	: 8 GB
Hard Drive	: 512GB
Camera	: HD Webcam (for gesture recognition)
Microphone	: Inbuilt/External (for voice commands)

3.2 SOFTWARE SPECIFICATIONS

Operating System	: Windows 11
Front-end Technology	: OpenCV, Mediapipe, PyAutoGUI
Back-end-Technology	: Python logic - SpeechRecognition.
IDLE/Editor	: pycharm/ VS code

3.3 ABOUT THE LANGUAGE

Python

Python is a high-level, interpreted, and general-purpose programming language widely used in software development, artificial intelligence, computer vision, and automation. It is known for its simplicity, readability, and large library support, making it ideal for both frontend (OpenCV, MediaPipe , PyAutoGUI) and backend backend (Speech recognition , logging) parts of the project.

Why python?

- Ease of Learning & Readability - Simple syntax close to English, easy for rapid development.

- Extensive Library Support - Provides powerful libraries like OpenCV, Mediapipe, pyAutoGUI, SpeechRecognition, and pyttsx3.
- Cross-Platform Compatibility - Works smoothly on windows, linux, and macOS.
- Integration Capability - Can integrate with other languages, APIs, and hardware easily.
- Rapid Prototyping - Supports faster development and testing for research and projects.

Advantages of Python

Versatile:

Supports multiple domains like AI, machine learning, computer vesion, and automation.

Strong support for AI/ML:

Rich frameworks for intelligent systems.

Open Source:

Free to use and continuously updated.

Productivity Boost:

Reduces development time compared to low-level languages.

Portability:

Programs can run without modifications on different systems.

Platform Independent:

Write once, run anywhere.

FRONT-END TECHNOLOGIES

Front-end technologies in HOVI-MOUSE are used to provide gesture-based control and user interaction. They ensure that the system captures hand movements and translates them into real-time mouse actions, creating a smooth and hands-free experience.

Required Libraries:

- Python-OpenCV
- Python-Mediapipe
- Python-PyAutoGUI

OpenCV (Open Source Computer Vision Library)

OpenCV is used to capture and process real-time video from the webcam. It detects hand gestures, tracks movement, and converts them into cursor actions such as moving, clicking, dragging, and scrolling.

Mediapipe:

Mediapipe provides powerful hand-tracking models that identify hand landmarks and gestures. It simplifies gesture recognition with high accuracy, allowing precise cursor control without physical devices.

PyAutoGUI:

PyAutoGUI executes the actual system-level mouse action. Based on detected gestures, it performs clicks, drags, scroll, and other controls, enabling seamless integration between user gestures and computer operations.

BACK-END TECHNOLOGIES

Back-end technologies in HOVI-MOUSE handle voice commands, system processing, and responses. They ensure that the user's speech is recognized, interpreted, and executed correctly to perform system-level actions.

Required libraries/Environment:

- Python Logic
- SpeechRecognition
- Command prompt

Python Logic:

Python acts as the backbone of the system, integrating gesture recognition, voice commands, and system control into a unified workflow. It also manages multitasking between gesture and voice inputs, ensuring smooth and parallel execution of user commands.

Speech Recognition:

This library processes user voice commands, converts speech to text, and triggers corresponding system actions (e.g., right click, press down, close window). It provides noise handling and casual speech support, making the system reliable in real-life environments.

Command prompt:

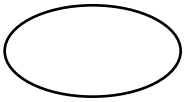
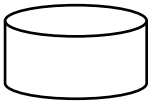
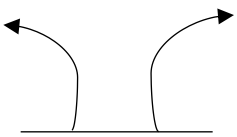
The command prompt acts as the backend environment for this project. It is used to execute the python program and run the system continuously in the background. Once the program is started through the command prompt, the virtual mouse remains active and can control any application on the system. In this way, the command prompt provides the execution platform for all voice and gesture-based commands, ensuring smooth interaction between the user and the operating system.

SYSTEM DESIGN

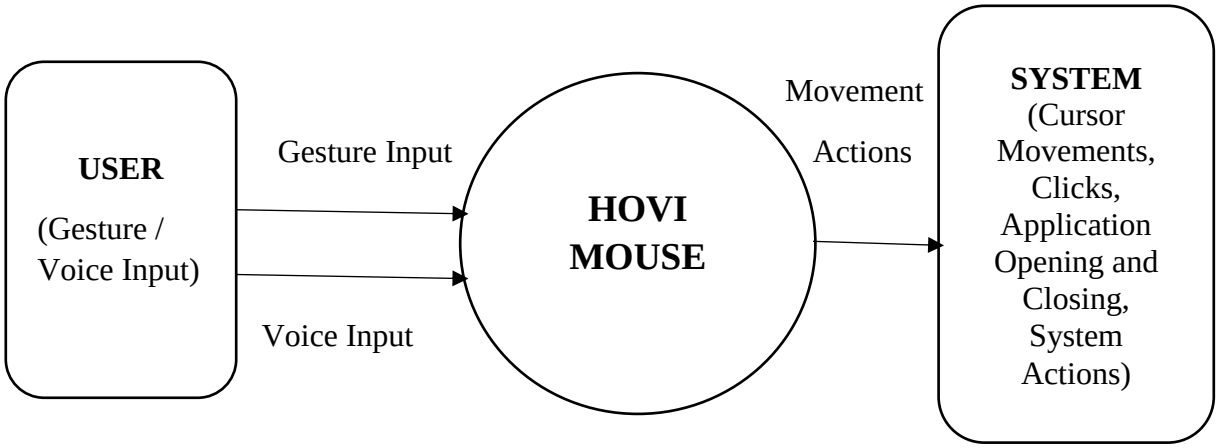
4. SYSTEM DESIGN

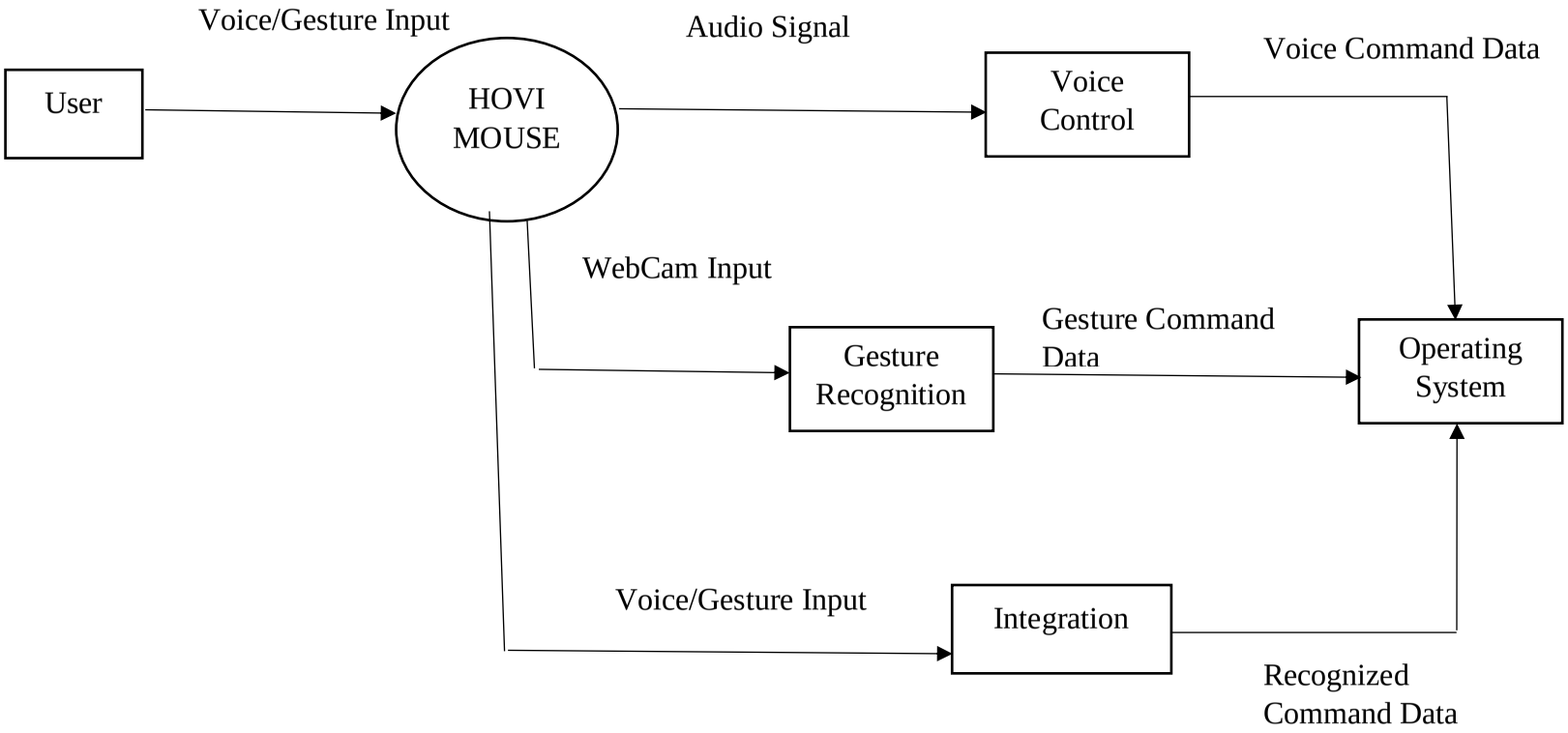
4.1 DATA FLOW DIAGRAM

The data flow diagram is a graphical tool. It is used to describe and analyze the movement of the data through a system manual or automated. The focus is on the Data flowing into the System between processed and in out of data stores. This is a central tool and basis from which other components are developed. The system models are termed as Data Flow diagrams. The Data flow Diagram is also known as a Data Flow Graph or a bubble chart.

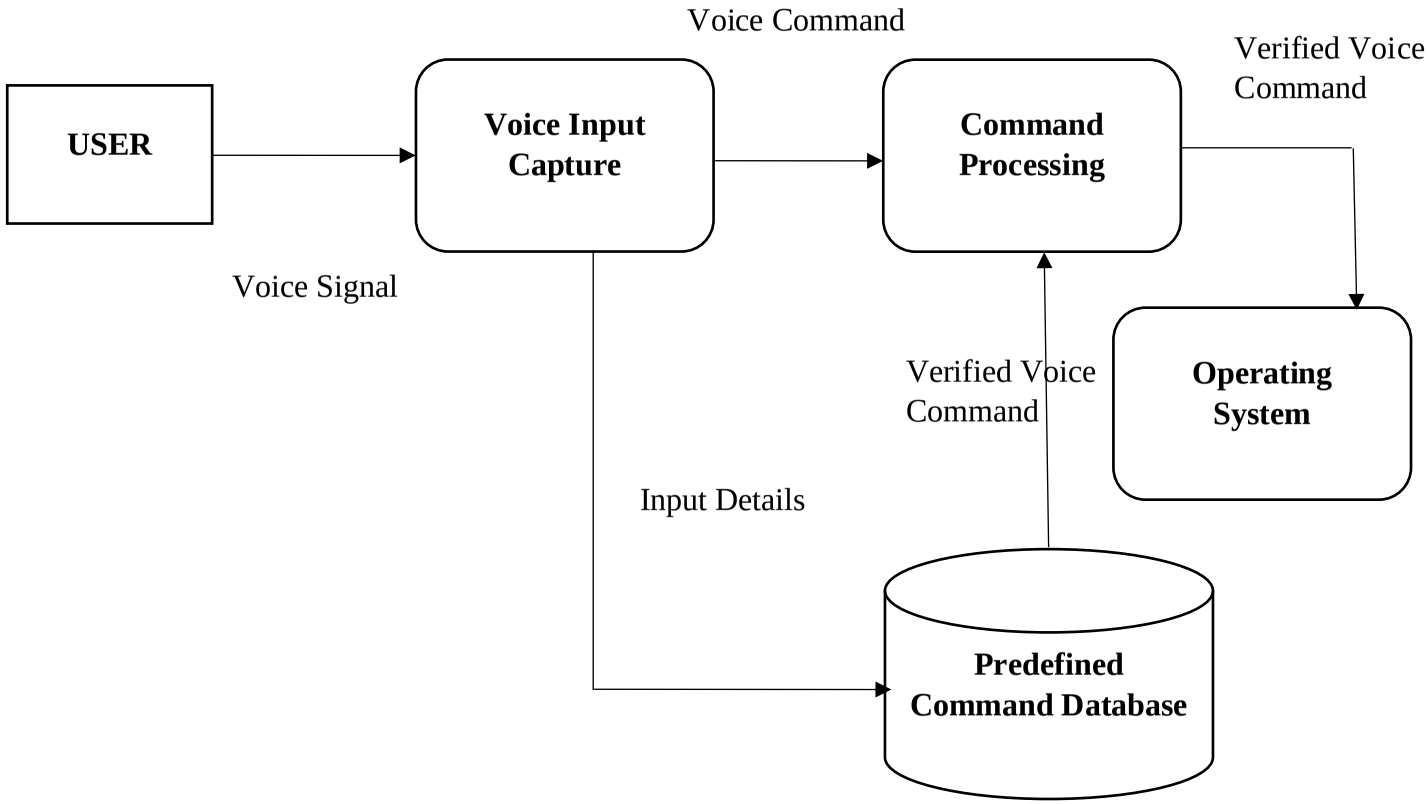
Symbol	Name	Function
	Data flow	Used to show the movement of data between processes, data stores and external entities. The arrowhead indicates the direction of data flow
	Process	Represents a transformations or operations performed on data; it takes input data and produces output data.
	Source of sink (External Entity)	Represents an external source or destination of data- an entity outside the system that provides input or receives output
	Data set	Represents a collection of related data used for training, testing or evaluation
	Data Store	A repository where data is stored for later use; data can be read or written by processes.

LEVEL 0 DFD

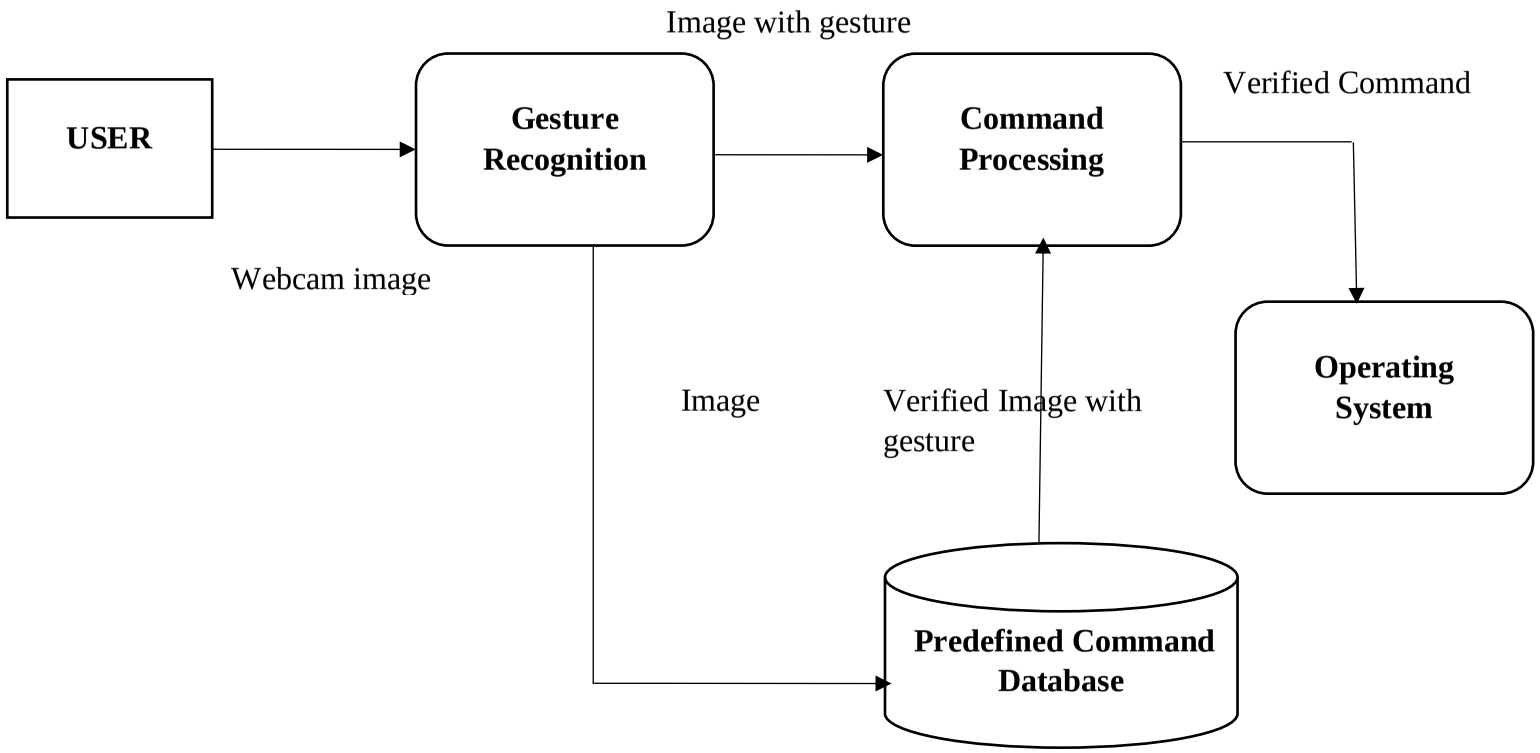

USER
(Gesture / Voice Input)
Gesture Input
Voice Input
HOVI MOUSE
Movement Actions
SYSTEM
(Cursor Movements, Clicks, Application Opening and Closing, System Actions)


Voice/Gesture Input
Audio Signal
Voice Command Data
User
HOVI
MOUSE
Voice
Control
WebCam Input
Gesture Command
Data
Gesture
Recognition
Operating
System
Voice/Gesture Input
Integration
Recognized
Command Data

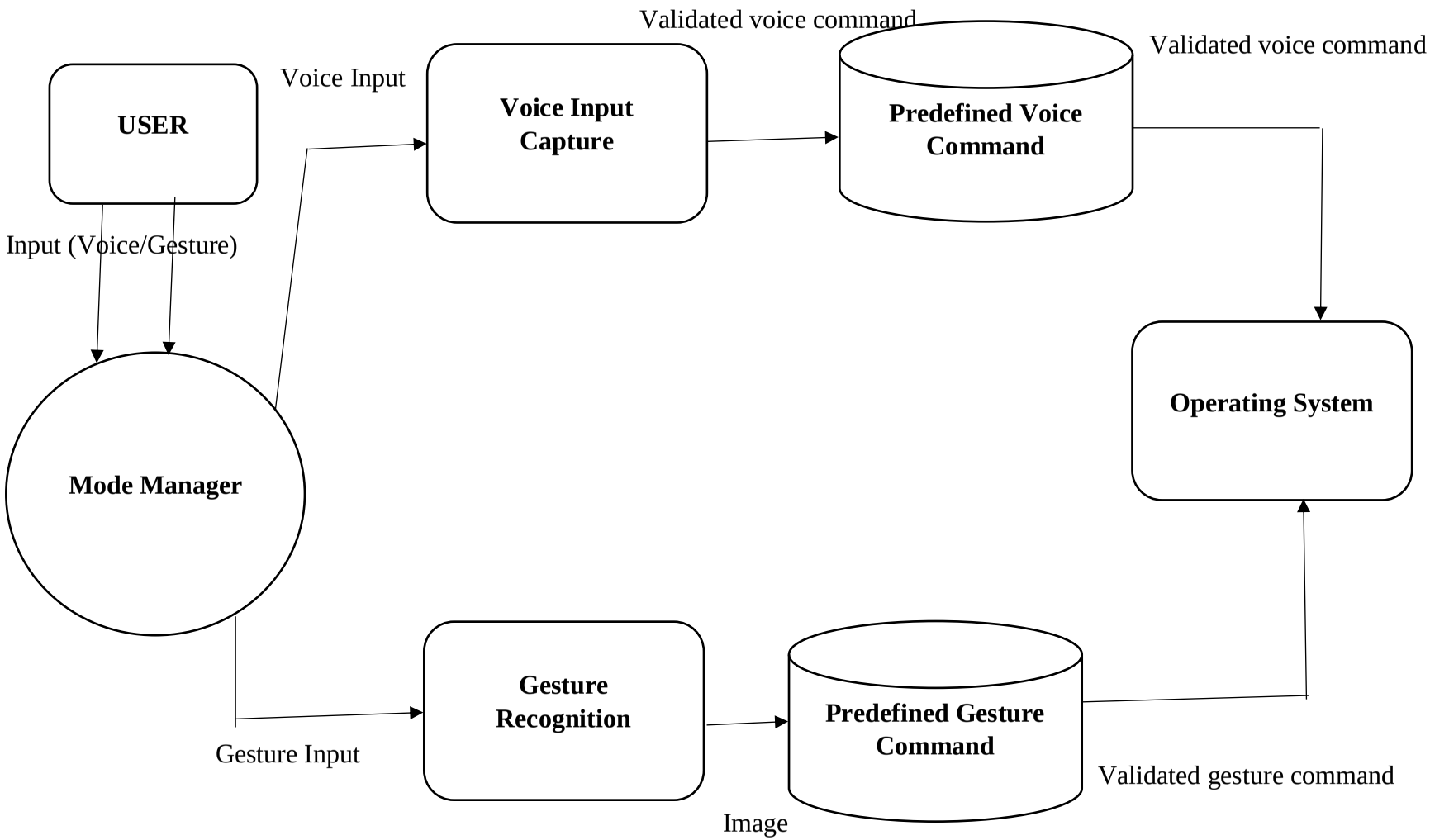
LEVEL 1 DFD



LEVEL 2 DFD VOICE CONTROL



LEVEL 2 DFD GESTURE MODULE



LEVEL 2 DFD INTEGRATION MODULE

4.2 MODULE DESCRIPTION

A module description provides a detailed explanation of the individual functional units of the project. Each module represents a specific part of the system, highlighting its purpose, working principle, and features. By describing modules separately, the project can be better understood in terms of its structure, functionality, and integration.

Gesture Control Module Description

The gesture control module uses the webcam along with OpenCV and Mediapipe to detect and track hand landmarks in real-time. It allows users to perform mouse functions through intuitive hand gestures. Supported actions include left click, right click, double click, scrolling, cursor movement and navigation across the screen. Gestures are also mapped to actions such as copy and paste. This module enables smooth, natural, and touch-free control of the system.

Hand Detection

The system uses a webcam to capture real-time hand movements. With the integration of OpenCV and MediaPipe, hand landmarks are accurately detected and tracked. This ensures smooth and reliable recognition of finger positions, enabling precise gesture-based control.

Mouse Actions via Gestures

Hand gestures are mapped to standard mouse functions such as left click, right click, double click and other basic events. Additional actions such as cursor movement, copy and paste are also supported. This module allows users to perform complete mouse operations without physical devices.

Navigation Control

Gestures enable seamless navigation across the desktop environment. Users can move the cursor, select files, and interact with applications efficiently, providing a fully hands-free computing experience

Voice Control Module Description

The Voice Command Module enables the system to perform mouse operations and application tasks through speech recognition. It processes spoken instructions and maps them to predefined system actions, allowing accurate and efficient execution.

This module supports both basic commands (for standard mouse functions) and advanced commands (for application management and text input). By reducing reliance on physical devices, it provides a completely hands-free interaction method, enhancing accessibility and user convenience.

Voice Command Input

The system uses a microphone to capture user speech and processes it through the SpeechRecognition library. Voice commands are converted into text and matched with predefined actions to ensure accurate execution.

System Control

The module supports both basic and advanced operations, enabling complete mouse functionality and application navigation through voice input. This hands-free control improves efficiency and accessibility while reducing dependency on traditional input devices.

Error Handling

To ensure high accuracy, background noise is filtered, and the system is designed to wait for clear inputs before execution. This minimizes misinterpretation of commands and enhances overall reliability.

System Integration Module Description

The combined control module is the core innovation of the project. It integrates both voice commands and gesture control to provide a seamless and flexible interaction experience. Users can switch between speech and gestures, or use both together, depending on convenience and context. This fusion enhances system usability, making it more efficient and adaptable for real-world applications, especially in accessibility and hands-free computing scenarios.

Unified Control

The integration module combines gesture-based and voice-based control into a single, seamless workflow. It allows users to switch naturally between hand gestures and voice commands, providing a flexible and intuitive interaction experience.

Error Recovery

The system incorporates safe exit and recovery mechanisms, enabling immediate termination or suspension of operations when required. It also manages potential conflicts when both gesture and voice inputs are active simultaneously, ensuring smooth and reliable functionality.

Performance Tracking

System performance is continuously monitored by logging the success and failure of executed commands. This evaluation helps assess overall accuracy, identify areas for improvement, and maintain reliability. By integrating gesture and voice control, the module provides a hybrid interaction experience that supports efficient multitasking and uninterrupted system operations.

Task Execution & Performance Analysis

The Task Execution and Performance Analysis module manages the execution of user tasks through gestures and voice commands and evaluates system performance. It allows users to perform actions freely, tracks accuracy and reliability, and provides feedback or suggestions to improve efficiency and interaction quality.

Gesture & Voice Task Execution

This module enables users to perform predefined tasks through both gesture and voice commands. It provides hands-free control over system operations, allowing users to interact naturally with the desktop environment and applications.

User - Controlled Timing

In HOVI-MOUSE, The system is fully user-driven, with no fixed time limits for task execution. Users can perform actions such as clicking, copying, pasting, opening applications, or using text input at their own pace and as many times as required. This flexibility enhances usability and accommodates individual workflow preferences.

Review & Retake

The Review and Retake allows users to monitor and improve their interaction with the system. It provides access to feedback logs for failed or misrecognized actions, enables users to retry commands at any time, and continuously tracks progress and accuracy to support effective learning and performance improvement.

Users can:

- Display feedback logs for failed or misrecognized actions.
- Allows users to retry commands at any time.
- Continuously tracks progress and accuracy.

SYSTEM IMPLEMENTATION

5. SYSTEM IMPLEMENTATION

Requirements:

Python – Version 3.10

OpenCV – Version 4.7.0

Mediapipe – Version 0.10.5

pyAutoGUI –Version ES6

SpeechRecognition – Version 3.10.0

Keyboard - Version 0.13.5

Explanation:

Python (3.10)

A high-level programming language that serves as the core for implementing gesture and voice recognition. Its extensive library support makes it ideal for AI-powered applications.

OpenCV (4.7.0)

A computer vision library used for real-time image processing. In our project, it captures and tracks hand movements to control the virtual mouse cursor smoothly.

Mediapipe (0.10.5)

A framework developed by Google for efficient hand landmark detection. It enables accurate finger tracking for gestures like click, drag, and scroll.

PyAutoGUI (0.9.54)

A cross- platform automation library used to simulate mouse and keyboard actions. It executes the recognized gestures and voice commands into real system-level operations.

SpeechRecognition (3.10.0)

A library for converting spoken voice commands into text. It allows the system to recognize commands like “right click”, “press down”, or “close window”.

Keyboard (0.13.5)

A python library that handles system-level key presses. In this project,it is used to perform actions like closing the current window (via “close window” command), sending ESC key events, and application-specific closing commands (e.g., “close chrome").

5.1 SAMPLE CODING

5.1.1 Voice commands

```
import speech_recognition as sr import pyautogui

class VoiceCommandSample:

def   init  (self):

self.recognizer = sr.Recognizer() self.microphone = sr.Microphone()

self.speech_controller = SpeechController() # For text mode

self.commands = {

# Basic Mouse Operations 'left click': self.left_click, 'right click': self.right_click,

'double click': self.double_click, 'scroll up': self.scroll_up,

'scroll down': self.scroll_down,

# Mouse Movement

'move left': lambda: self.move_mouse(-50, 0), 'move right': lambda: self.move_mouse(50, 0),

'move up': lambda: self.move_mouse(0, -50), ‘move down': lambda: self.move_mouse(0, 50), # Drag Operations

'start drag': self.start_drag, 'release': self.release_drag, # Application Control

'open chrome': self.open_chrome, 'open notepad': self.open_notepad, 'open calculator': self.open_calculator, 'close chrome': self.close_chrome, 'close notepad': self.close_notepad,

# Text Mode Toggle
```

'text mode on': lambda: self.speech_controller.enable_text_mode(), 'text mode off': lambda: self.speech_controller.disable_text_mode(),

}

def process_voice_command(self, command):

"""Process voice command - handles both mouse and text modes""" command

= command.lower().strip()

Check if text mode is active first

if self.speech_controller.is_text_mode_active():

if self.speech_controller.process_speech(command): return True

Process mouse commands

if command in self.commands: self.commands[command]()

return True return False

def left_click(self): pyautogui.click()

print("Voice: Left Click executed")

def right_click(self): pyautogui.rightClick()

print("Voice: Right Click executed")

def double_click(self): pyautogui.doubleClick()

print("Voice: Double Click executed")

def scroll_up(self): pyautogui.scroll(3)

print("Voice: Scroll Up executed")

def scroll_down(self): pyautogui.scroll(-3)

print("Voice: Scroll Down executed")

def move_mouse(self, x_offset, y_offset): current_x, current_y = pyautogui.position()

pyautogui.moveTo(current_x + x_offset, current_y + y_offset) print(f"Voice: Mouse moved ({x_offset}, {y_offset})")

def start_drag(self): pyautogui.mouseDown() print("Voice: Drag started")

def release_drag(self): pyautogui.mouseUp() print("Voice: Drag released")

def open_chrome(self): import subprocess

subprocess.Popen(['chrome.exe']) print("Voice: Chrome opened")

def open_notepad(self): import subprocess

subprocess.Popen(['notepad.exe']) print("Voice: Notepad opened")

def open_calculator(self): import subprocess

subprocess.Popen(['calc.exe']) print("Voice: Calculator opened")

def close_chrome(self):

Import subprocess

subprocess.run(['taskkill', '/F', '/IM', 'chrome.exe']) print("Voice: Chrome closed")

def close_notepad(self): import subprocess

subprocess.run(['taskkill', '/F', '/IM', 'notepad.exe'])

print("Voice: Notepad closed")

def listen_for_commands(self): """Main listening loop"""

print("Voice Commands Ready. Say commands like:") print("- 'left click', 'right click', 'double click'") print("- 'scroll up', 'scroll down'") print("- 'move left', 'move right', 'move up', 'move down'") print("- 'open chrome', 'close chrome'")

print("- 'text mode on' (enables text input)") print("- 'text mode off' (disables text input)")

while True: try:

with self.microphone as source:

audio = self.recognizer.listen(source, timeout=1)

command = self.recognizer.recognize_google(audio) print(f"Heard: '{command}'")

if self.process_voice_command(command): print("Command executed successfully")

else:

print("Command not recognized")

except sr.UnknownValueError:

Pass

except KeyboardInterrupt: print("Voice commands stopped") break

if name == " main ":

voice_controller = VoiceCommandSample() voice_controller.listen_for_commands()

Explanation of Voice Command Sample Code

The given code demonstrates the implementation of a Voice Command Module for controlling mouse operations and applications using the Python language. It integrates speech recognition with automation libraries to execute user-defined commands in real time.

Libraries Used

- **Speech_recognition (sr):** Captures and processes voice input from the microphone.
- **Pyautogui:** Automates mouse operations such as clicking, scrolling, dragging, and cursor movement.
- **Subprocess:** Executes system-level commands to open or close applications like Chrome, Notepad, and Calculator.

Class Definition

The code defines a class VoiceCommandSample, which contains:

- A recognizer (sr.Recognizer()) and microphone (sr.Microphone()) for capturing voice input.

- A dictionary of predefined commands mapped to functions, allowing flexible control over mouse actions, application management, and text mode operations.
- A speech controller instance (SpeechController) for enabling and disabling text input mode.

Command Processing

- The function process_voice_command() handles the recognition and execution of commands.
- If text mode is active, spoken input is processed and directly printed as text.
- If text mode is not active, the command is matched against the predefined dictionary. If a match is found, the corresponding function is executed.

Mouse Control Functions

The module provides mouse functionalities such as:

- **Click Operations:** Left click, right click, and double click.
- **Scrolling:** Scroll up and scroll down.
- **Movement:** Moves the cursor relative to its current position.
- **Drag and Release:** Simulates drag-and-drop functionality.

Application Control

The code can open and close applications using system commands:

- **Open Applications:** Chrome, Notepad, and Calculator.
- **Close Applications:** Chrome and Notepad are closed using task termination commands.

Listening Loop

- The function listen_for_commands() continuously listens for user input.
- Captured audio is converted into text using Google Speech Recognition.
- If a valid command is recognized, it is executed; otherwise, feedback is provided to indicate that the command was not recognized.
- The loop runs until the user manually stops it with a keyboard interrupt (Ctrl+C).

Error Handling

- **UnknownValueError:** Handles cases where speech is unclear or unrecognizable.
- **KeyboardInterrupt:** Allows safe termination of the program when needed.

Program Execution

- The main block (if name == “ main ":) creates an instance of the class and starts the command listening process.

5.1.2 Gesture Commands

import cv2

import mediapipe as mp import math

import time import ctypes

from collections import deque

Windows API for direct input user32 = ctypes.windll.user32

MOUSEEVENTF_LEFTDOWN = 0x0002 MOUSEEVENTF_LEFTUP = 0x0004
MOUSEEVENTF_RIGHTDOWN = 0x0008 MOUSEEVENTF_RIGHTUP = 0x0010
MOUSEEVENTF_WHEEL = 0x0800

class GestureCommandSample:

def init (self):

Initialize MediaPipe

self.mp_hands = mp.solutions.hands

self.mp_drawing = mp.solutions.drawing_utils

Camera setup

self.cap = cv2.VideoCapture(0) self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)

self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480) # Gesture detection parameters

self.touch_threshold = 0.08

self.gesture_buffer = deque(maxlen=5) self.cursor_buffer = deque(maxlen=3)

Action cooldowns self.last_action_time = 0

self.action_cooldown = 0.5 # State

self.is_dragging = False

self.screen_width = user32.GetSystemMetrics(0) self.screen_height = user32.GetSystemMetrics(1)

def calculate_distance(self, p1, p2):

"""Calculate distance between two points"""

return math.sqrt((p1.x - p2.x)**2 + (p1.y - p2.y)**2) def get_finger_states(self, landmarks):

"""Get finger up/down states (1=up, 0=down)""" fingers = []

Thumb - horizontal check

thumb_is_up = landmarks[4].x > landmarks[3].x fingers.append(1 if thumb_is_up else 0)

Other fingers - vertical check

finger_tips = [8, 12, 16, 20] # Index, Middle, Ring, Pinky finger_pips = [6, 10, 14, 18] #PIP joints

for tip, pip in zip(finger_tips, finger_pips):

finger_is_up = landmarks[tip].y < landmarks[pip].y fingers.append(1 if finger_is_up else 0)

return fingers

def detect_touch_gesture(self, landmarks):

"""Detect thumb touching other fingertips""" thumb_tip = landmarks[4]

fingertips = [

landmarks[8], # Index finger landmarks[12], # Middle finger landmarks[16], # Ring finger landmarks[20] # Pinky finger

]

distances = [self.calculate_distance(thumb_tip, tip) for tip in fingertips] min_distance = min(distances)

closest_finger = distances.index(min_distance) if min_distance < self.touch_threshold:

touch_gestures = ["left_click", "right_click", "double_click", "press_enter"] return touch_gestures[closest_finger]

return "none"def detect_finger_pattern(self, landmarks): """Detect finger pattern gestures""" fingers = self.get_finger_states(landmarks)

Define gesture patterns patterns = {

(0, 1, 0, 0, 0): "cursor_move", # Index finger only

(0, 1, 1, 0, 0): "scroll_up", # Peace sign(Index + Middle)

(0, 0, 1, 1, 0): "scroll_down", # Middle + Ring fingers

(1, 0, 0, 0, 1): "copy", # Call Me gesture (Thumb + Pinky)

(1, 0, 0, 1, 0): "paste", # Thumb + Ring finger

(0, 0, 0, 0, 0): "drag_start", # Closed fist

(1, 1, 1, 1, 1): "drag_release", # Open hand

}

pattern = tuple(fingers)

return patterns.get(pattern, "none")

def detect_gesture(self, landmarks):

"""Main gesture detection - prioritize touch over patterns""" # First check touch gestures

touch = self.detect_touch_gesture(landmarks) if touch != "none":

return touch

Then check finger patterns

return self.detect_finger_pattern(landmarks)

def stabilize_gesture(self, raw_gesture):

"""Stabilize gesture detection to avoid false triggers"""

self.gesture_buffer.append(raw_gesture)

if len(self.gesture_buffer) >= 3:

recent = list(self.gesture_buffer)[-3:] # Count occurrences

counts = {} for g in recent:

counts[g] = counts.get(g, 0) + 1

Get most common gesture

most_common = max(counts.keys(), key=lambda x: counts[x]) # Require at least 2 out of 3 detections

if counts[most_common] >= 2:

return most_common return "none"

def smooth_cursor(self, x, y):

"""Smooth cursor movement""" self.cursor_buffer.append((x, y))

if len(self.cursor_buffer) >= 2:

avg_x = sum(pos[0] for pos in self.cursor_buffer) / len(self.cursor_buffer) avg_y = sum(pos[1] for pos in self.cursor_buffer) / len(self.cursor_buffer) return int(avg_x), int(avg_y)

return int(x), int(y)

def execute_gesture(self, gesture, cursor_x, cursor_y):

"""Execute gesture actions""" current_time = time.time()

```
# Handle cursor movement (no cooldown needed) if gesture == "cursor_move":

smooth_x, smooth_y = self.smooth_cursor(cursor_x, cursor_y)
user32.SetCursorPos(smooth_x, smooth_y)

Return

# Check cooldown for other actions

if current_time - self.last_action_time < self.action_cooldown: return

32

 try:

 if gesture == "left_click" and not self.is_dragging:
user32.mouse_event(MOUSEEVENTF_LEFTDOWN, 0, 0, 0, 0)

 time.sleep(0.05)

user32.mouse_event(MOUSEEVENTF_LEFTUP, 0, 0, 0, 0) print("Gesture: Left Click")

self.last_action_time = current_time


elif gesture == "right_click" and not self.is_dragging:
user32.mouse_event(MOUSEEVENTF_RIGHTDOWN, 0, 0, 0, 0)

time.sleep(0.05)

user32.mouse_event(MOUSEEVENTF_RIGHTUP, 0, 0, 0, 0) print("Gesture: Right Click")

self.last_action_time = current_time

elif gesture == "double_click" and not self.is_dragging:

# Double click user32.mouse_event(MOUSEEVENTF_LEFTDOWN, 0, 0, 0, 0)

user32.mouse_event(MOUSEEVENTF_LEFTUP, 0, 0, 0, 0) time.sleep(0.1)

user32.mouse_event(MOUSEEVENTF_LEFTDOWN, 0, 0, 0, 0)
```

user32.mouse_event(MOUSEEVENTF_LEFTUP, 0, 0, 0, 0) print("Gesture: Double Click")

self.last_action_time = current_time

elif gesture == "press_enter":

user32.keybd_event(0x0D, 0, 0, 0) # VK_RETURN time.sleep(0.05) user32.keybd_event(0x0D, 0, 0x0002, 0) # Key up print("Gesture: Press Enter") self.last_action_time = current_time

elif gesture == "scroll_up":

user32.mouse_event(MOUSEEVENTF_WHEEL, 0, 0, 120, 0) print("Gesture: Scroll Up")

33

self.last_action_time = current_time

elif gesture == "scroll_down": user32.mouse_event(MOUSEEVENTF_WHEEL, 0, 0, -120, 0)

print("Gesture: Scroll Down") self.last_action_time = current_time

elif gesture == "copy":

Ctrl+C

user32.keybd_event(0x11, 0, 0, 0) # Ctrl down

user32.keybd_event(0x43, 0, 0, 0) # C down time.sleep(0.05) user32.keybd_event(0x43, 0, 0x0002, 0) # C up user32.keybd_event(0x11, 0, 0x0002, 0) # Ctrl up print("Gesture: Copy")

self.last_action_time = current_time elif gesture == "paste":

Ctrl+V

user32.keybd_event(0x11, 0, 0, 0) # Ctrl down

user32.keybd_event(0x56, 0, 0, 0) # V down time.sleep(0.05) user32.keybd_event(0x56, 0, 0x0002, 0) # V up user32.keybd_event(0x11, 0, 0x0002, 0) # Ctrl up print("Gesture: Paste")

self.last_action_time = current_time

elif gesture == "drag_start" and not self.is_dragging: smooth_x, smooth_y = self.smooth_cursor(cursor_x, cursor_y) user32.SetCursorPos(smooth_x, smooth_y)

user32.mouse_event(MOUSEEVENTF_LEFTDOWN, 0, 0, 0, 0) self.is_dragging = True print("Gesture: Drag Start")

self.last_action_time = current_time

elif gesture == "drag_release" and self.is_dragging: user32.mouse_event(MOUSEEVENTF_LEFTUP, 0, 0, 0, 0)

self.is_dragging = False

print("Gesture: Drag Release")

self.last_action_time = current_time except Exception as e:

print(f"Gesture execution error: {e}")

def run_gesture_detection(self):

"""Main gesture detection loop""" print("Gesture Commands Ready:")

print("- Touch thumb to index finger = Left Click") print("- Touch thumb to middle finger = Right Click") print("- Touch thumb to ring finger = Double Click") print("- Touch thumb to pinky = Press Enter")

print("- Point with index finger = Move cursor") print("- Peace sign (index + middle) = Scroll Up") print("- Middle + Ring fingers = Scroll Down") print("- Call Me gesture (thumb + pinky) = Copy")

print("- Thumb + Ring finger = Paste") print("- Closed fist = Drag Start") print("- Open hand = Drag Release")

with self.mp_hands.Hands(max_num_hands=1, min_detection_confidence=0.7, min_tracking_confidence=0.7

) as hands:

while True

ret, frame = self.cap.read() if not ret:

break

```
# Mirror frame for natural interaction frame = cv2.flip(frame, 1)

h, w, _ = frame.shape

# Convert to RGB for MediaPipe

rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) results = hands.process(rgb_frame)

current_gesture = "none"

if results.multi_hand_landmarks:

hand_landmarks = results.multi_hand_landmarks[0] # Draw hand landmarks
self.mp_drawing.draw_landmarks(

frame, hand_landmarks, mp.solutions.hands.HAND_CONNECTIONS)

# Detect gesture

raw_gesture = self.detect_gesture(hand_landmarks.landmark)

stable_gesture = self.stabilize_gesture(raw_gesture) if stable_gesture != "none":

current_gesture = stable_gesture

# Get cursor position from index finger index_tip = hand_landmarks.landmark[8]

cursor_x = index_tip.x * self.screen_width cursor_y = index_tip.y * self.screen_height # Execute
gesture

self.execute_gesture(stable_gesture, cursor_x, cursor_y)

# Display current gesture

cv2.putText(frame, f"Gesture: {current_gesture}",   (10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0,

255, 0), 2)

cv2.imshow('Gesture Control', frame) if cv2.waitKey(1) & 0xFF == ord('q'):

break
```

Cleanup

if self.is_dragging: user32.mouse_event(MOUSEEVENTF_LEFTUP, 0, 0, 0, 0

)

self.cap.release() cv2.destroyAllWindows()

if name == " main ":

gesture_controller = GestureCommandSample() gesture_controller.run_gesture_detection()

Explanation of Gesture commands (sample code)

This program lets you control your computer using hand gestures captured by a webcam. It uses MediaPipe to detect hand movements, OpenCV to capture and display video, and Windows API (via ctypes) to perform actions like mouse clicks, scrolling, and keyboard shortcuts.

Libraries Used

- cv2 – handles video input and output.
- mediapipe – detects and tracks hand landmarks.
- math – calculates distances between points (used for gesture detection).
- time – sets cooldowns so actions don't repeat too fast.
- ctypes – connects with Windows system functions to simulate mouse/keyboard events.
- deque – smooths gestures and cursor movement.

Class Setup

When the class starts:

- Opens the camera (640x480 resolution)
- Sets up MediaPipe Hand Tracking.
- Defines a touch threshold for gestures like thumb-to-finger taps.
- Prepares buffers to stabilize gestures and cursor movement.
- Defines a cooldown time (0.5s) between actions.
- Tracks whether the mouse is currently dragging.
- Reads the screen size to move the cursor correctly.

Gesture Detection

The program detects gestures in two ways:

a. Touch Gestures (thumb touching fingers):

- Thumb + Index → Left Click
- Thumb + Middle → Right Click
- Thumb + Ring → Double Click
- Thumb + Pinky → Press EnteR

b. Finger Patterns (based on raised/lowered fingers):

- Index only → Move Cursor
- Index + Middle → Scroll Up
- Middle + Ring → Scroll Down
- Thumb + Pinky → Copy (Ctrl+C)
- Thumb + Ring → Paste (Ctrl+V)
- Closed fist → Start Drag
- Open hand → Release Drag

Stabilization

- Gestures can flicker due to camera noise.
- The program uses a buffer (last 3 gestures) and only confirms a gesture if it appears at least 2 times.
- Cursor positions are also averaged across frames for smooth movement.

Executing Gestures

Once a gesture is recognized:

- Cursor gestures → move the mouse pointer.
- Click gestures → left, right, or double click.
- Enter gesture → simulates pressing the Enter key.
- Scroll gestures → scroll up or down.

- Copy/Paste gestures → simulate Ctrl+C and Ctrl+V.
- Drag gestures → hold and release mouse button Cooldown ensures the same action doesn't repeat too quickly.

Main Loop

- Continuously captures video frames.
- Flips the frame for natural interaction (mirror effect).
- Detects hand landmarks and draws them on the screen.
- Recognizes gestures, executes actions, and shows the current gesture on the display.
- Stops if the user presses 'q'.

Program Execution

- The main block (if name == " main ":) creates an instance of the class.
- It starts the gesture detection or voice command listening process.
- The system then runs continuously, processing user inputs in real time until the program is stopped.

SYSTEM TESTING

6. SYSTEM TESTING

6.1 SYSTEM TESTING

System testing is a crucial phase in the Software Development Life Cycle (SDLC) where the entire integrated system is tested to verify that it meets specified requirements and functions correctly in the intended environments. It is performed after unit testing and integration testing and before acceptance testing. The primary goal of system testing in this project is to validate that the HOVI MOUSE: Gesture and Voice-Controlled Virtual Mouse functions as a complete solution by accurately recognizing gestures, executing voice commands, and controlling system operations smoothly. This testing ensures that the platform behaves as intended and provides reliable, hands-free computing experience to the users.

6.2 TYPES OF TESTING

System testing for this project involves verifying the functionality, performance, security and usability of the entire HOVI MOUSE system, including the voice recognition module, gesture detection system, camera integration, microphone processing, and real-time input control mechanisms. The testing approach focuses on two comprehensive testing methodologies:

- Unit Testing
- Performance Testing
- Integration Testing

1. Unit Testing

Unit testing is the process of testing individual components or modules of a system independently to ensure they perform their intended functions correctly. In the HOVI MOUSE project, unit testing is applied to both the voice command and gesture recognition modules to validate their accuracy and reliability. For example, the VoiceMouseController and WorkingGestureController classes are tested for correct execution of commands and gesture recognition without interference from other modules.

Voice Command Module Testing – VoiceMouseController Class

Voice command module testing focuses on validating that the system correctly interprets and executes spoken commands. In HOVI MOUSE, this includes testing microphone detection, initialization, ambient noise adjustment, and audio input quality to ensure reliable command capture. For instance, the microphone is tested to confirm it can detect input and filter background noise effectively, allowing commands like “move left” or “open Chrome” to execute smoothly.

Basic Movement Commands

Basic movement command testing ensures that directional commands move the cursor accurately on the screen. In HOVI MOUSE, commands like “move left” or “move down” are tested for a fixed pixel movement, such as 50 pixels in the indicated direction, to verify correct response. This confirms that simple cursor operations respond precisely to user voice inputs.

Example:

- “Move left” - 50px leftward movement.
- “ Move right” - 50px rightward movement
- “Move up “ - 50px upward movement
- “ Move down” - 50px downward movement

Precision Movement Commands

Precision movement testing focuses on finer cursor control, ensuring small or large movements respond accurately. For example, in HOVI MOUSE, the command “move left small” moves the cursor 10 pixels, while “move right big” moves 100 pixels, with accuracy maintained within a 2-pixel tolerance. This allows users to perform delicate operations, such as clicking small icons or dragging files precisely.

Example:

- “Move left small” - 10px precise movement
- “Move right big” - 100px large movement
- Movement accuracy within 2px tolerance

Click Operation Testing

Click operation testing verifies that mouse button actions triggered by voice commands execute correctly. In HOVI MOUSE, commands like “left click,” “right click,” and “double click” are tested to ensure single or rapid double-click execution occurs reliably, enabling standard mouse functionality without physical input.

Example:

- “Left click” - Single left mouse button press
- “ Right click” - Context menu activation
- “Double click” - Rapid double-click execution

Scroll Operations Testing

Scroll operation testing confirms that vertical page navigation commands work accurately. In HOVI MOUSE, commands such as “scroll up,” “page down,” or “page up” are validated to ensure smooth scrolling and full-page navigation using only voice inputs.

Example:

- “Scroll up” - Upward page scrolling
- “ Scroll down” - Downward page scrolling

Drag and Drop Operations

Drag and drop testing ensures the system can perform complex mouse operations using sequential commands. In HOVI MOUSE, commands like “start drag,” “release,” and “drop” are tested to simulate holding, moving, and releasing objects on the screen correctly.

Applications Launch and Termination Commands

Application command testing verifies that voice commands can open and close programs accurately. In HOVI MOUSE, commands like “open Chrome,” “open Notepad,” or “close Calculator” are tested to ensure correct process initiation and termination, including proper cleanup after closing.

Example:

- “Opennotepad” - Notepad application launch
- “Open chrome” - Chrome browser launch
- “Close notepad” - Notepad process termination
- “Close chrome” - Chrome process termination
- “Close calculator” - Calculator process termination Process cleanup verification

Text Mode Integration

Text mode integration testing validates the activation and deactivation of speech-to-text functionality. In HOVI MOUSE, “text mode on” activates the SpeechController, while “text mode off” deactivates it, with persistence checked across sessions to ensure stable operation.

Example:

- “ text mode on” - SpeechController activation
- “ text mode off” - SpeechController deactivation Mode state persistence

Command Logging and Error Handling

Testing of command logging and error handling ensures that every command execution is accurately recorded with timestamps and console outputs. Invalid commands are processed safely without crashing the system, maintaining reliability and providing appropriate user feedback.

Example:

- Timestamp generation for each command
- Console output formatting
- Command execution status tracking

SpeechController Class Testing

The SpeechController is tested for number recognition, text command processing, mode toggling, and status reporting. In HOVI MOUSE, spoken numbers like “one” or “fifty-six” are converted to numeric values accurately, while commands like “new line” or “tab” simulate keyboard events.

Mode activation, state change notifications, and session persistence are verified to ensure consistent behavior.

Gesture Recognition Module Testing – WorkingGestureController Class

Gesture recognition testing validates that hand and finger movements are correctly detected and translated into system actions. In HOVI MOUSE, camera initialization, hand landmark detection, and finger state recognition are tested to confirm accurate cursor control and gesture execution. For example, index finger movement controls the cursor, while specific combinations like thumb + index perform clicks.

Touch and Pattern Recognition Testing

Touch and pattern recognition testing ensures specific hand gestures trigger the intended actions. Commands like thumb + index for left click, or closed fist for copy, are tested for accuracy and stability. Gesture thresholds and confidence levels are validated to minimize errors.

Example:

- Indexfingeronly -Cursor movement
- Middle + Ring + Pinky - Scroll down
- Close fist – Copy
- Open palm - Paste

Cursor Movement and Gesture Stabilization

Testing cursor movement and stabilization verifies smooth, accurate tracking and reduces false positives. In HOVI MOUSE, noise filtering and gesture buffers ensure that cursor motion is precise and gestures remain stable over time.

DirectInputSystem Class Testing

Direct input testing validates low-level mouse and keyboard operations. In HOVI MOUSE, coordinate-based movements, click operations, scroll actions, and keyboard inputs like Ctrl+C and Ctrl+V are tested for accuracy across multiple screen resolutions and monitors.

2. Performance Testing

Performance testing measures system efficiency, such as voice recognition latency (target under 500ms), gesture detection latency (target under 200ms), camera frame rate (28 FPS average), RAM usage (145MB average), and CPU utilization (22% average). This ensures HOVI-MOUSE responds quickly without overloading system resources.

3. Integration Testing

Integration testing verifies that multiple modules work together correctly and interact seamlessly with applications. In HOVI-MOUSE, voice and gesture inputs are tested for simultaneous operation, smooth mode switching, rapid context switching, error handling, and resource sharing between camera and microphone. Integration tests also ensure correct interaction with external applications like Chrome, File Explorer, and WhatsApp. All integration tests confirm that the system provides a consistent and reliable user experience.

Voice-Gesture Integration Testing

Voice-Gesture integration testing ensures that voice commands and gesture inputs work together seamlessly. In HOVI-MOUSE, simultaneous inputs are managed by giving priority to voice commands while synchronizing states to prevent conflicts. Mode switching between text mode and gesture control is smooth, with rapid context changes handled without data loss. The system also gracefully recovers from invalid commands and efficiently shares resources like the camera and microphone.

System-Application Integration Testing

This testing verifies that HOVI-MOUSE interacts correctly with external applications. Commands like “open File Explorer,” “Open Chrome,” and “Open WhatsApp” are tested for navigation, document editing, and messaging using both voice and gesture inputs. It ensures a consistent and reliable hands-free experience across multiple applications.

Example:

File Explorer Integration

- Voice command: “open file explorer” File selection and manipulation Folder navigation

Web Browser Integration

- Voice command : “Open chrome voice activated browser launch Gesture-based link clicking Document editing operations

Whatsapp Integration

- Voice command : “ Open Whatsapp” Launches Whatsapp
- Voice-based text input for messaging Gesture scroll through chats

Testing Environment and Summary Testing Tools:

- **Unit Testing:** Python unit test framework Performance Monitoring: psutil and time modules Testing Result:
- **Integration Tests:** All test executed -PASSED

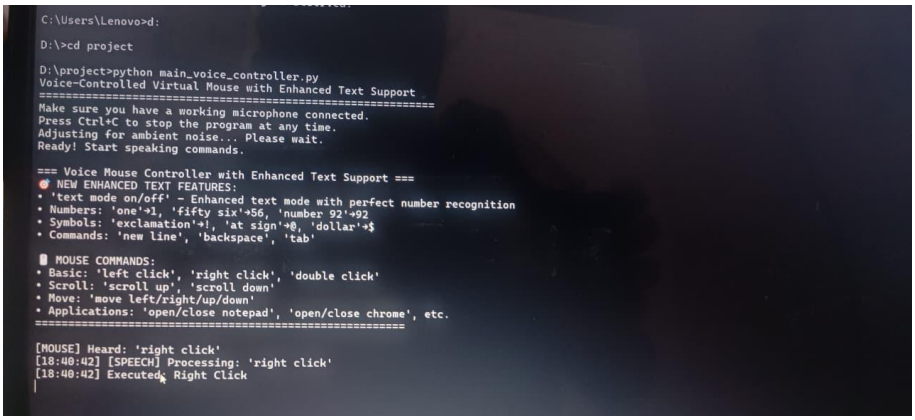
Testing Environment and Summary

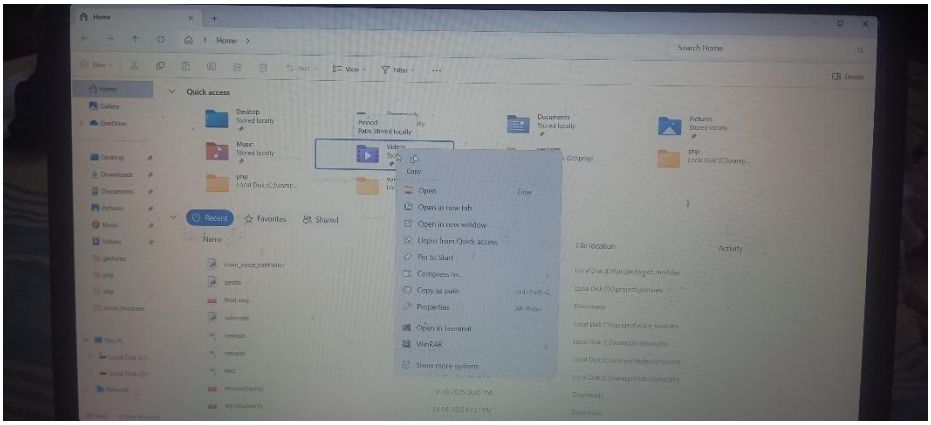
Unit testing is performed using the Python unit test framework, with performance monitored via psutil and time modules. Integration tests confirm that all modules and system interactions function correctly, and all tests were executed successfully, demonstrating the reliability and efficiency of the HOVI MOUSE system.

SCREENSHOTS

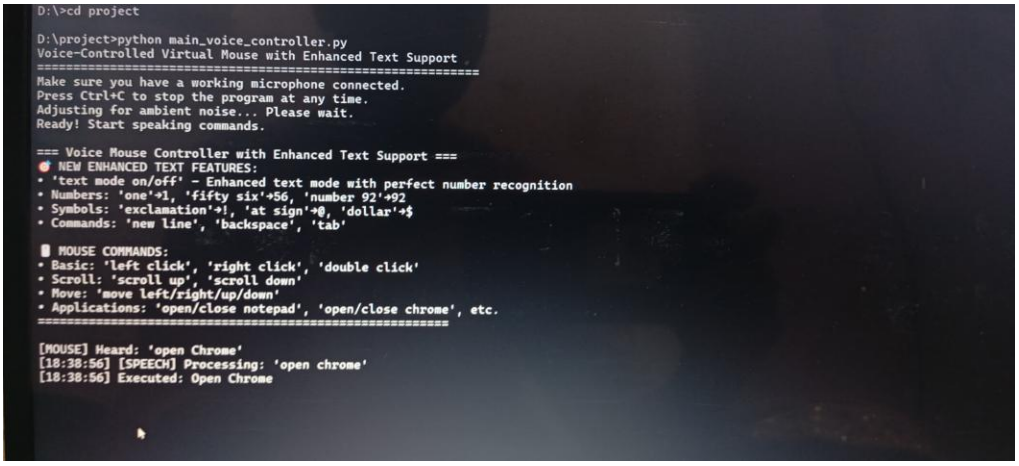
VOICE COMMANDS:

EXECUTION OF RIGHT CLICK:

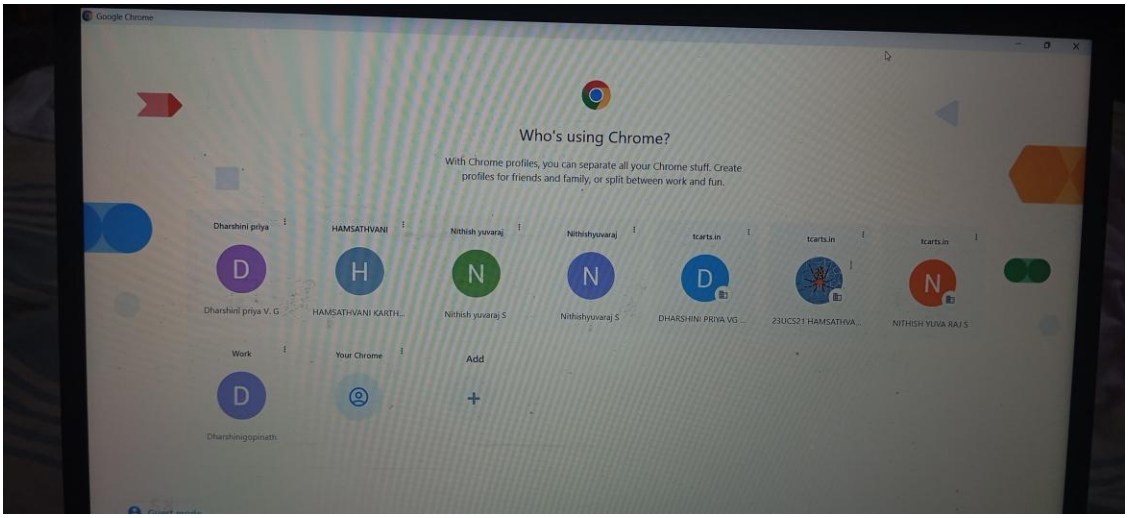




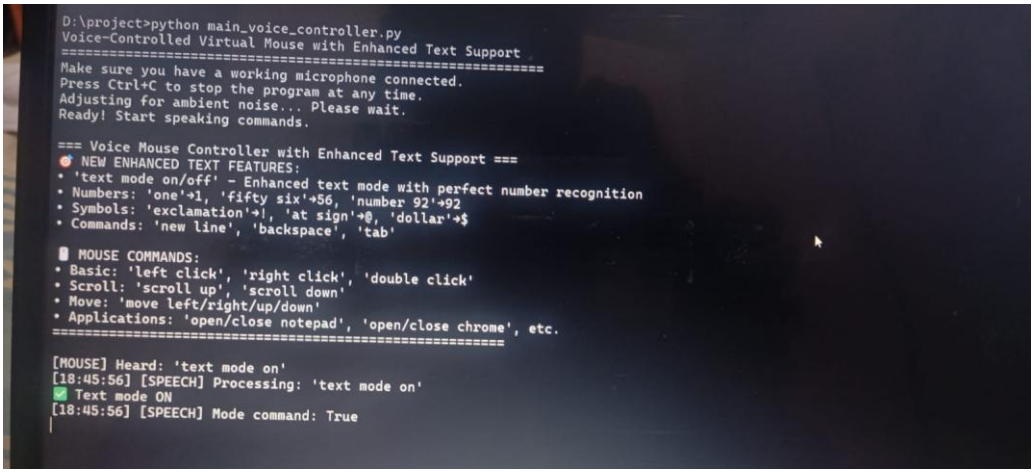
OPENING OF APPLICATION(CHROME):



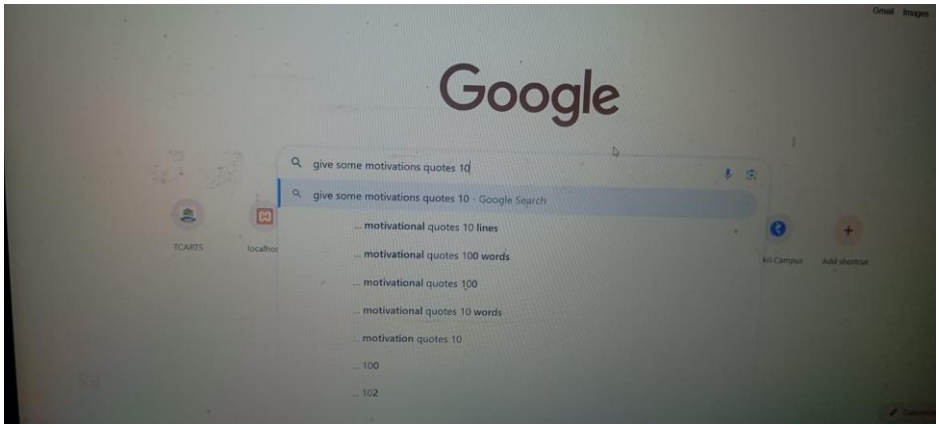
(OPENING OF CHROME APPLICATION):



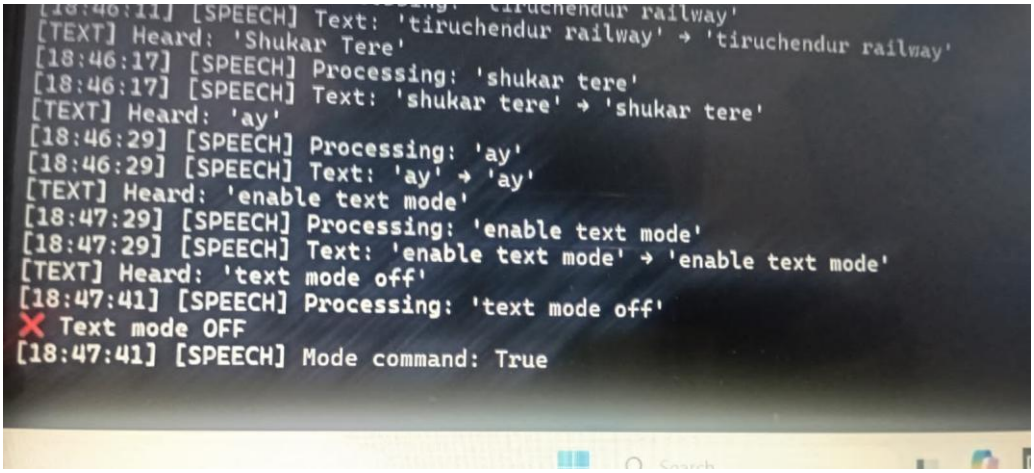
ENABLING TEXT MODE



(TYPING THE WORDS SAID)

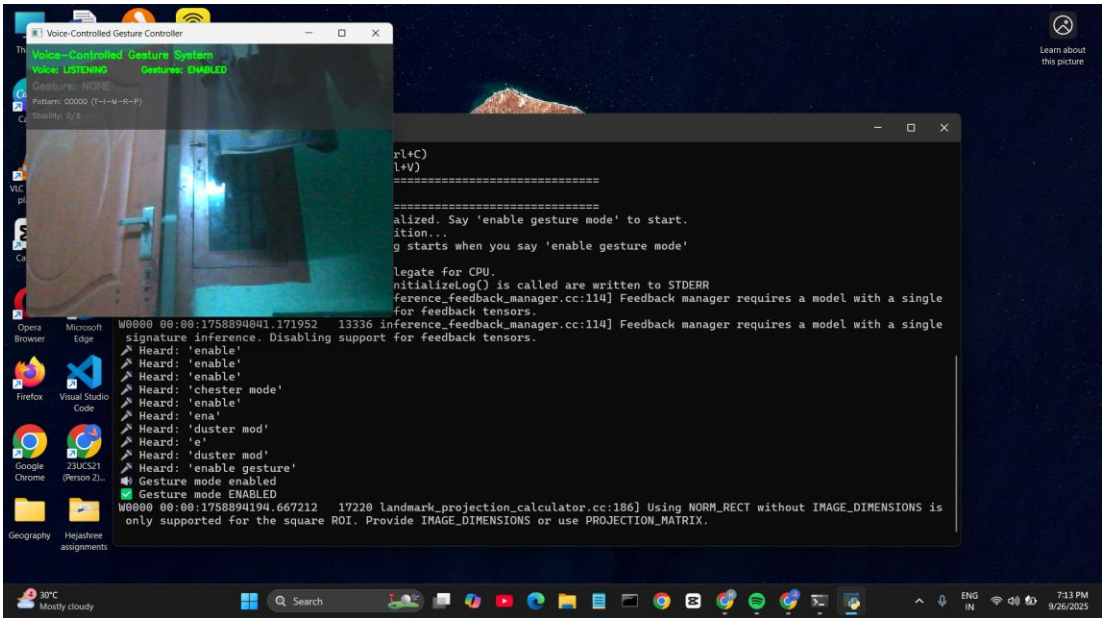


DISABLING TEXT MODE:

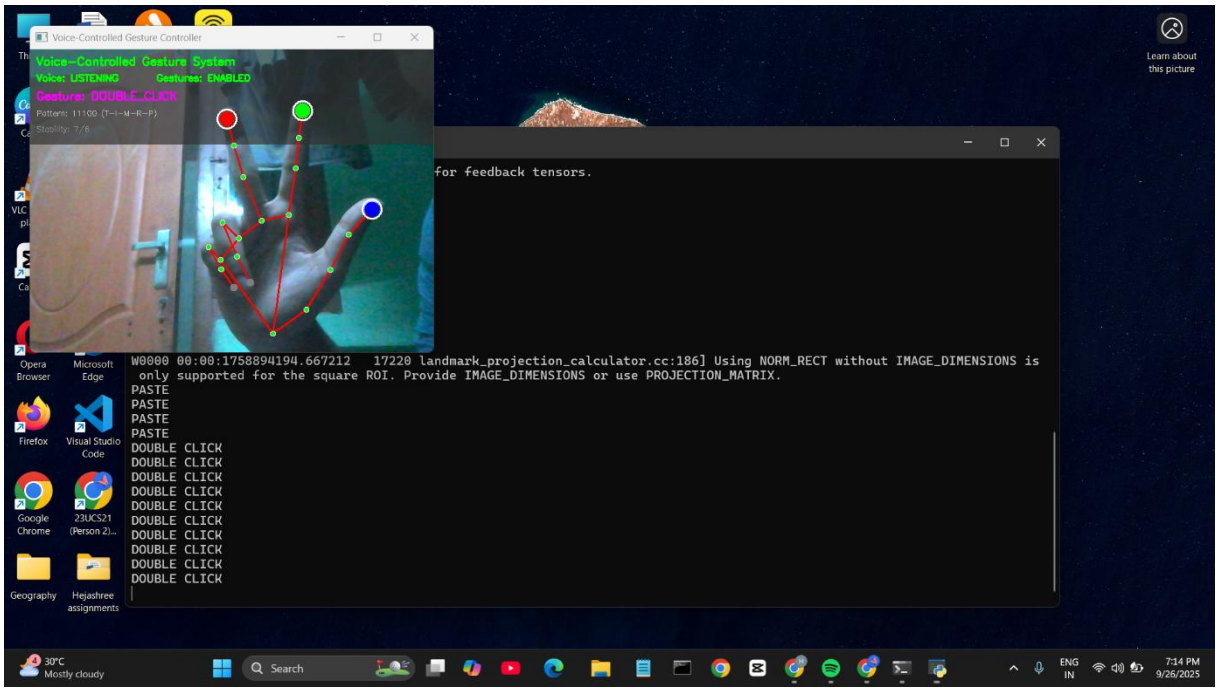


GESTURE COMMANDS

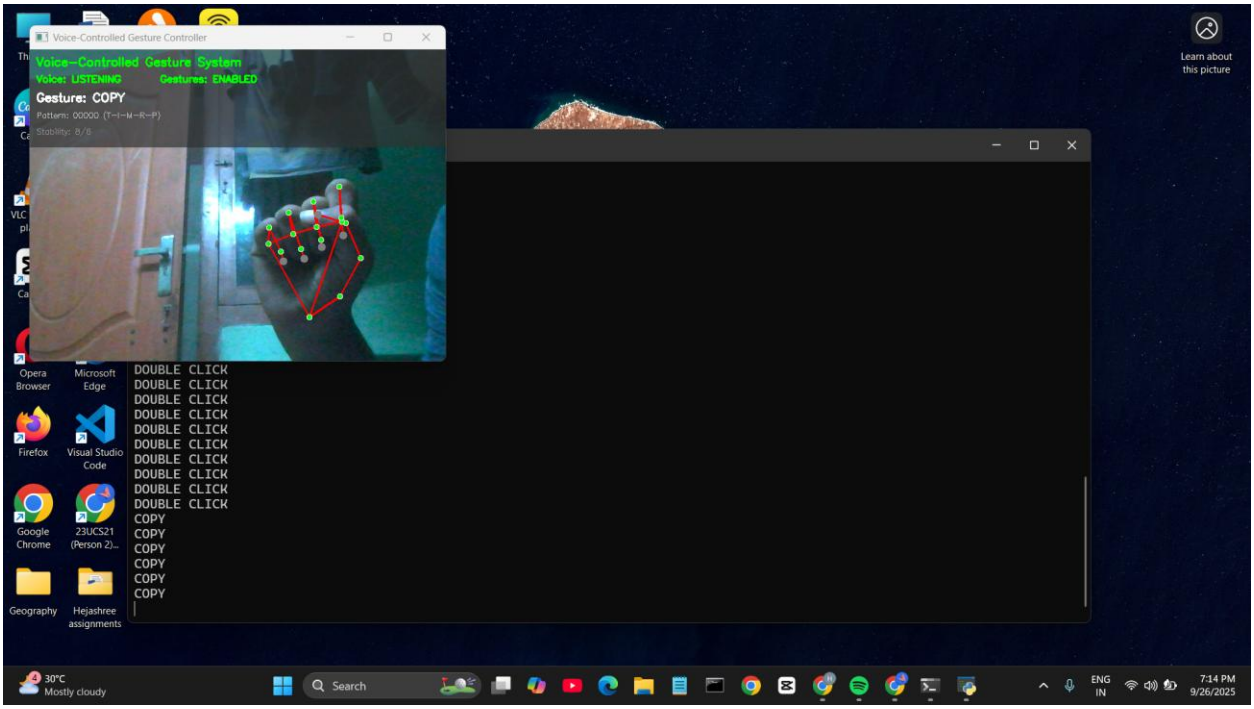
ENABLING GESTURE MODE:



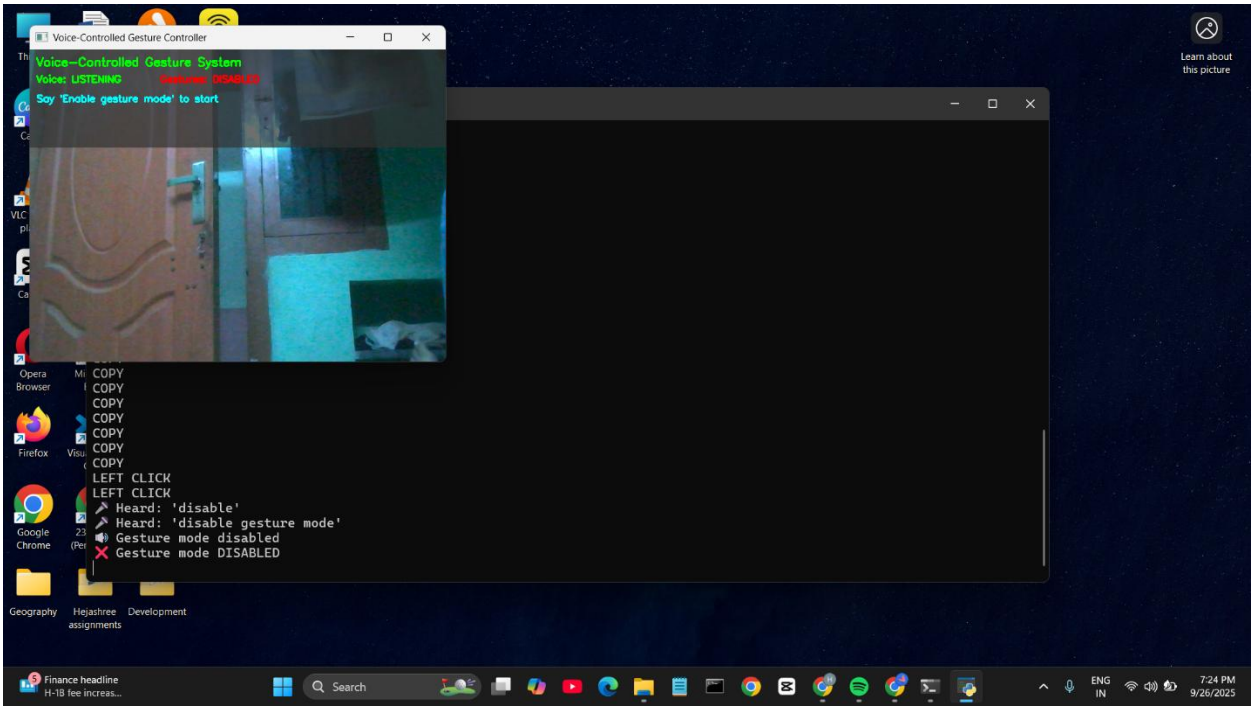
EXECUTION OF DOUBLE CLICK:



EXECUTION OF COPY COMMAND:



DISABLING OF GESTURE MODE:



CONCLUSION

8. CONCLUSION

The HOVI MOUSE is an AI-based system that allows users to control the computer in an easy and accessible way. It works through hand gestures using OpenCV and mediapipe, and voice commands using SpeechRecognition, pyautoGUI, and Keyboard automation.

The system supports all major mouse functions like move, left click,right click,double click, drag and drop, and scrolling. It also provides advanced features such as dropdown navigation (press up,press down, press enter), closing windows.

With noise-tolerant voice recognition and smooth gesture tracking. The platform ensures accurate and user-friendly performance. It also responds calmly to casual commands and allows users to customize settings for better control.

This project makes computer usage hands-free, efficient, and accessible, supporting modern human-computer interaction and helping users perform daily tasks in a smarter way.

FUTURE ENHANCEMENTS

9. FUTURE ENHANCEMENTS

Future enhancement refers to the process of improving an existing system by adding new capabilities or refining current functions to make it more efficient, user-friendly, and suitable for real-world use. It focuses on upgrading the performance, usability, and interaction of the system to meet future technological needs.

FEATURE IMPROVEMENTS

Multi-language Voice Command Support

The future version of the HOVI-MOUSE can be improved to support multiple languages such as Tamil, English, and others. This allows users to control the system in their preferred language, making it more flexible and accessible. For example, users can say commands like “Left click,” “Open Chrome,” or “Move right” either in English or Tamil. This enhancement helps people from different linguistic backgrounds to use the system easily.

Smart and Natural Voice Interaction

Instead of responding only to fixed commands, the enhanced version will be able to understand casual and natural speech. For example, if a user says “please right click here” or “can you open Chrome,” the system will still recognize the action and perform it correctly. This makes the HOVI-MOUSE more intelligent, interactive, and closer to human-like communication, similar to virtual assistants such as Google Assistant or Alexa.

Standalone Application without Python Setup

In the enhanced version, the HOVI-MOUSE will be available as a standalone executable application, which can be downloaded and used directly without installing Python or any additional setup. Users can simply click a download button and run the application smoothly on their system. This feature makes the system more practical, user-friendly, and ready for real-world usage, improving accessibility for all types of users.

.

BIBLIOGRAPHY

BIBLIOGRAPHY

BOOK REFERENCE

a. Gary Bradski & Adrian Kaehler, Learning OpenCV 3: Computer Visioning C++ with the openCV Library ,O'Reilly Media, 2017.

b. Daniel Jurafsky & James H. Martin, Speech and Language Processing, Pearson, 2019.

c. Steven Bird, Ewan Klein & Edward Loper, Natural Language Processing with Python, O'Reilly Media, 2009

WEB REFERENCE

https://opencv.org/

https://google.github.io/mediapipe/

https://pypi.org/project/SpeechRecognition/

https://pyautogui.readthedocs.io/

https://pypi.org/project/keyboard/

https://www.python.org/doc/